

Daniel Kalo, Pranjal Rane
CS 6120
Summer 2024
Final Project

Development of a Clinical Chatbot for Patient Support Using NLP

Introduction

The healthcare industry is experiencing rapid advancements in technology, particularly in artificial intelligence and natural language processing (NLP). One of the key applications of these technologies is the development of intelligent chatbots that can assist patients with healthcare-related queries, in order to avoid adhering to clinic or doctor hours, and be able to find accurate responses and diagnoses in real time. The goal of this project is to create a clinical chatbot that can understand and respond to patient inquiries, provide medication information, diagnose patients, and offer basic medical advice, including which medicine to take, based on symptoms described by the users.

Detailed Components

1. Data Collection and Preprocessing

Objective: To collect and preprocess datasets for training the chatbot.

Tasks:

- **Data Collection:** Four different datasets were gathered from publicly available sources such as medical forums, QA websites, and clinical databases.

Specifically:

- Two comprehensive datasets of healthcare-related questions and answers.
 - Two detailed datasets of diseases and drugs based on common symptoms.
- **Preprocessing:**
 - The data underwent several preprocessing steps including tokenization, stop word removal, and text normalization (stemming/lemmatization). This ensured that the dataset was clean and ready for training.
 - However, some models, such as BERT and GPT-2 ended up producing more inaccurate responses with the preprocessed/tokenized data in comparison to the raw data. This is likely due to the uniqueness of words used in the medical space.

Implementation:

- Tokenization was done using libraries like NLTK and SpaCy.
- Stop words were removed to eliminate non-informative words, and stemming/lemmatization was applied to normalize the text data.

2. Intent Recognition and Entity Extraction

Objective: To develop models that accurately recognize user intents and extract relevant entities from the user queries.

Tasks:

- **Intent Recognition:** SpaCy was used to build and train models for recognizing user intents. These intents included symptom checking, medication information, condition information, treatment information, genetic information, research updates, diagnosis information, and possible risk factors.
- **Entity Extraction:** Models were trained to identify and extract relevant entities from the text. These entities included symptoms, medication names, and dates.

Implementation:

- SpaCy's pre-trained models were fine-tuned to detect specific healthcare-related entities.
- Custom entity extraction models were developed using transfer learning on existing datasets.

3. Response Generation

Objective: To implement a system that generates accurate and relevant responses to user queries.

Tasks:

- **Rule-Based System:** A rule-based system was developed for straightforward queries, such as appointment scheduling.
- **Neural-Based System:** A neural-based response generation model was implemented for more complex queries. Models like GPT-2, Random Forest and BERT were used to generate responses.

- **Database Integration:** The system was integrated with a medical information database to ensure accurate and reliable responses.

Implementation:

- The BERT model 'bert-large-uncased-whole-word-masking-finetuned-squad' was utilized for the question-answering tasks.
- The final 'chat' function ties together the different components, handling user input, determining the intent, extracting entities, and generating responses based on the identified context.

4. Dialogue Management

Objective: To create a dialogue management system that can handle multi-turn conversations.

Tasks:

- **State Machine/Neural Dialogue Manager:** A state machine or neural dialogue manager was implemented to keep track of the conversation context and handle multi-turn interactions effectively.
- **Fallback Mechanisms:** Fallback mechanisms were developed to handle unknown queries and guide the conversation back on track.

Implementation:

- The dialogue management system was implemented using a state machine that handles different states such as query identification, response generation, and feedback collection.
- Fallbacks were designed to prompt the user for clarification or provide default responses when the chatbot could not determine an appropriate answer.

5. Evaluation and User Testing

Objective: To evaluate the performance and usability of the chatbot.

Tasks:

- **User Testing:** Conducted user testing sessions to gather feedback on the chatbot's usability and effectiveness.
- **Evaluation Metrics:** The chatbot was evaluated using metrics such as precision, recall, and F1-score for intent recognition. User satisfaction scores were also collected.
- **Error Analysis:** Performed error analysis to identify and address common issues in the chatbot's responses.

Implementation:

- User feedback was collected through surveys and interviews.
- Precision, recall, and F1-score metrics were computed to evaluate the accuracy of intent recognition and entity extraction.

Methodology

The project was executed in several phases, starting with data collection and preprocessing, followed by model development for intent recognition, entity extraction, and response generation. Finally, the dialogue management system was implemented and the chatbot was thoroughly evaluated through user testing and performance metrics.

Key Functions and Implementation Details

- **'generate_responses_bert_v2' Function:** This function uses the BERT model to generate answers to user queries. It tokenizes the input text, runs it through the BERT model, and extracts the most probable answer from the context provided.
- **'recommend_drugs' Function:** This function is part of the drug recommendation system. It takes a user's query and returns a list of recommended drugs by matching the query against a database of drug information.
 - The methodology involved several steps, including data preprocessing, text vectorization, model training, and the serialization of the model for future use:

1. Data Preprocessing

The initial step was loading the cleaned drug review dataset, which had been preprocessed to remove noise, irrelevant data, and any missing values. The dataset primarily contained textual reviews of drugs and their associated ratings. The focus was on the 'review' column, which comprised free-text reviews of

various drugs. This textual data served as the foundation for the recommendation system.

2. Text Vectorization

Given that the reviews were in text form, it was necessary to convert them into a numerical representation that the machine learning model could interpret. For this purpose, we employed the Term Frequency-Inverse Document Frequency (TF-IDF) vectorization technique. The TF-IDF approach is effective in highlighting the importance of specific words in a document relative to a collection of documents (corpus). By removing common English stop words and focusing on unique words within the reviews, the TF-IDF vectorizer transformed the textual data into a matrix of TF-IDF features, which then served as the input for the model.

3. Model Training

To build the recommendation system, we utilized the Nearest Neighbors algorithm, specifically with the cosine distance metric and brute-force search method. This model was chosen for its efficiency in finding the most similar reviews based on the TF-IDF features. The model was trained on the TF-IDF matrix, enabling it to learn the relationships and similarities between different drug reviews.

4. Model Serialization

After training the model, we serialized both the TF-IDF vectorizer and the Nearest Neighbors model using Python's pickle module. This step ensures that the trained model and vectorizer can be saved to disk and later reloaded for making predictions on new data without retraining, thus enhancing the system's usability and efficiency in real-world applications.

- **'predict_disease' Function:** This function is part of the disease prediction system. It takes a user's query and returns the most probable disease based on the user symptoms.
 - The methodology encompasses data preparation, model training, and the serialization of the trained model for future use:

1. Data Preparation

The dataset utilized for this project was the "Cleaned Disease Symptom and Patient Profile Dataset," which contains various features related to patient symptoms and demographics, as well as the target variables 'Disease' and 'Outcome Variable.' Initially, the data was loaded into a pandas DataFrame and inspected for any inconsistencies or missing values.

2. Target Variable Encoding

Since the target variables ('Disease' and 'Outcome Variable') are categorical, they needed to be converted into a numerical format for machine learning algorithms to process them. This was achieved using Label Encoding. Separate label encoders were applied to each target variable to transform the categorical labels into integer representations.

3. Data Splitting

To evaluate the performance of the model, the dataset was split into training and testing sets. The 'train_test_split' function from 'scikit-learn' was employed to split the data, with 80% of the data used for training and 20% reserved for testing.

The random state was set to ensure reproducibility of results.

4. Feature Scaling

Given the different scales of the input features, it was necessary to standardize the data using StandardScaler. This step transformed the features to have a mean of 0 and a standard deviation of 1, which is essential for the effective performance of many machine learning algorithms, particularly those involving distance measurements.

5. Model Development

An ensemble learning approach was adopted by combining four different classifiers—Random Forest, Gradient Boosting, Support Vector Classifier (SVC), and K-Nearest Neighbors (KNN)—using a Voting Classifier. The voting classifier was configured for soft voting, which means the predicted probabilities from each classifier were averaged to make the final prediction. This ensemble model was then wrapped in a MultiOutputClassifier to handle the multi-output nature of the problem, enabling simultaneous prediction of both 'Disease' and 'Outcome Variable.'

6. Model Training and Evaluation

The multi-output model was trained on the training set, and predictions were made on the test set. The performance of the model was evaluated by calculating the accuracy and generating classification reports for each target variable separately. These metrics provided insights into how well the model performed in predicting the disease and its outcome.

7. *Model Serialization*

Finally, to facilitate future use of the model without needing to retrain, the trained multi-output model, label encoders, and scaler were serialized using Python's pickle module. This allows for the model to be saved to disk and reloaded at a later time for making predictions on new data.

- **'chat' Function:** The 'chat' function is the core of the chatbot's interaction system. It handles user inputs, determines the type of query (intent), extracts necessary entities, and generates a response. It integrates the entire pipeline from input processing to response delivery.
 - The 'chat' function is designed to be extensible, allowing for additional intents and responses to be added as needed.

Results

- **Data Preprocessing:** Successfully processed a dataset consisting of thousands of healthcare-related questions and answers.

- **Model Performance (BERT):** The intent recognition and entity extraction models achieved extremely high accuracy, with precision, recall, and F1-score metrics indicating robust performance.
- Evaluating balanced intent distribution:

predicted_intent	
Condition_Information	5505
Treatment_Information	3061
Symptom_Checking	2770
Genetic_Information	2607
Research_Updates	2252
Diagnosis_Information	846
Risk_Factors	356

- Evaluating Precision, Recall, F1-Score, and Support metrics for trained BERT intent-labeling model:

	precision	recall	f1-score	support
Genetic_Information	1.00	1.00	1.00	16
Condition_Information	0.94	1.00	0.97	17
Symptom_Checking	1.00	1.00	1.00	15
Research_Updates	1.00	1.00	1.00	8
Treatment_Information	1.00	1.00	1.00	11
Risk_Factors	1.00	1.00	1.00	14
Diagnosis_Information	1.00	0.92	0.96	12
accuracy			0.99	93
macro avg	0.99	0.99	0.99	93
weighted avg	0.99	0.99	0.99	93

- **Model Performance (Random Forest):**
 - Disease Prediction:

micro avg	0.26	0.26	0.26	70
macro avg	0.11	0.11	0.11	70
weighted avg	0.29	0.26	0.27	70

Disease Variable

accuracy			0.73	70
macro avg	0.73	0.73	0.73	70
weighted avg	0.74	0.73	0.73	70

Outcome Variable

- Drug Recommendation:

```
Accuracy: 0.90
Precision: 0.90
Recall: 0.90
F1-score: 0.89
```

- **User Feedback:** User testing (by us, the project creators) provided valuable feedback, which was used to refine the chatbot's functionality. The chatbot generated accurate and logical responses, therefore seeming promising regarding its usability and effectiveness in providing healthcare-related information.

Discussion

The project demonstrated the feasibility and effectiveness of using NLP techniques to develop a clinical chatbot. The integration of both rule-based and neural-based approaches allowed for a comprehensive solution capable of handling a wide range of user queries. However, challenges such as handling ambiguous queries, training BERT, Random Forest, and GPT-2 models that took a lot of time and computational resources (with a lot of trial and error), and integrating real-time data sources were identified and remain areas for future improvement.

Conclusion

This project successfully developed a clinical chatbot that leverages NLP to assist patients with healthcare-related queries. The chatbot is capable of understanding and responding to patient inquiries, providing accurate information, and managing basic tasks such as disease diagnosis, drug recommendations, and responding to any other medically related questions. The real-world application of this chatbot offers valuable insights into the potential of NLP solutions in healthcare that other chatbots do not solely focus on yet.

Instructions

Prerequisites:

- Python 3.11 installed on your system.
- Necessary Python libraries, including 'transformers', 'torch', 'SpaCy', 'sklearn', and others required for NLP tasks.
- Download Models:

- Download the necessary pre-trained BERT models and SpaCy ('en_core_web_lg' and/or 'en_core_web_lg') models if not already included.
- Prepare the Data:
 - Ensure the dataset for training and testing is properly placed in the designated directory.

Running the Chatbot:

1. Navigate to Chatbot directory

```
cd Chatbot
```

2. Install the libraries:

```
pip install -r requirements.txt
```

3. Start the Chatbot:

```
python chatbot.py
```

4. Interaction:

- Use the 'chat' function to interact with the chatbot
- We have tried our best for dialogue management however it fails sometimes, just rerun the python script to restart.

Deliverables

- **Original Datasets:** Folders and/or files of the original datasets that were used for preprocessing and training of our models.
- **Code:** A Jupyter notebook containing all 5 parts of the project, including their functions and their documentation.
- **Report:** This detailed report explaining the methodology, results, discussions, instructions, and other useful information for running the program successfully.
- **Presentation:** A set of slides summarizing the project for a class presentation.
- **Chatbot Application:** A functional chatbot implementation that can be run on any shell or terminal using Python.

Justification

This project involved multiple aspects of NLP including data preprocessing, intent recognition, entity extraction, response generation, and dialogue management. The comprehensive approach ensured a robust learning experience and provided practical exposure to developing NLP solutions for healthcare.

Dataset References

1. MedQuAD: <https://github.com/abachaa/MedQuAD>
2. COVID-Q: <https://github.com/deepset-ai/COVID-QA>
3. Disease Symptoms and Patient Profile:
<https://www.kaggle.com/datasets/uom190346a/disease-symptoms-and-patient-profile-dataset>
4. UCI ML Drug Review:

<https://archive.ics.uci.edu/dataset/462/drug+review+dataset+drugs+com>